

GitHub Profile Insight

Selenium Automation Testing Project

Technology: Java + Selenium WebDriver + TestNG

1. About the Website

GitHub Profile Insight is a web application designed to provide GitHub-related profile insights and features.

The application includes GitHub user search and modules for Git Rating, Trending, Compare, Learn Git, Settings and Login.

Website: <https://insight-eight-psi.vercel.app/>

Testing Objective

To automate important UI flows, verify page navigation, validate user search behavior, and confirm that major modules open correctly.

2. Modules Automated

Module	Automation Covered
Dashboard	Page opening and main heading verification
GitHub User Search	Valid and invalid username search
Git Rating	Navigation and URL verification
Trending	Navigation and URL verification
Compare	Navigation and URL verification
Learn Git	Direct navigation and URL verification
Settings	Navigation and URL verification
Login	Login button visibility/clickability verification

3. Tools & Technologies

- **Java** – Programming language used for automation scripts.
- **Selenium WebDriver** – Automates browser actions and web elements.
- **TestNG** – Test execution, priorities, and assertions.
- **ChromeDriver** – Controls the Chrome browser.
- **WebDriverWait** – Waits for elements and page conditions.
- **XPath** – Locates web elements using attributes, text, and relationships.

4. Why I Used XPath

XPath was used to identify web elements when unique IDs or other simple locators were not available.

Examples:

- `//input[contains(@placeholder,'Search GitHub users')]`
- `//*[contains(text(),'Git Rating')]`
- `//button[contains(.,'Login')]`

Using **contains()** makes the locator flexible when the complete attribute value or text may vary.

5. Waits & Assertions

Explicit Wait: WebDriverWait is used to wait until an element becomes visible/clickable or a URL condition is satisfied.

Example: visibilityOfElementLocated(), elementToBeClickable(), urlContains()

Assertions: TestNG Assert is used to verify expected results, such as whether an element is displayed or a page URL contains the expected path.

This improves synchronization and helps prevent failures caused by elements loading late.

6. Login Module – Current Scope

The current automation verifies the Login button is present and clickable.

It does **not** automate complete login authentication because valid test credentials and the expected authenticated behavior are required.

Future login scenarios:

- Valid username + valid password
- Invalid username/password
- Blank username/password
- Password field validation
- Successful login and logout verification

7. TestNG Structure

@BeforeTest – Opens Chrome, maximizes the browser, configures waits, and launches the application.

@Test(priority = n) – Executes individual module test methods in priority order.

Assert – Validates expected results.

@AfterTest – Closes the browser using driver.quit().

This structure keeps setup, test execution, validation, and cleanup organized.

8. Sample Automation Flow

1. Launch Chrome browser
2. Open GitHub Profile Insight website
3. Verify Dashboard heading
4. Search a valid GitHub user such as 'torvalds'
5. Search an invalid GitHub username and verify the response
6. Navigate through Git Rating, Trending, Compare, Learn Git and Settings
7. Verify Login button
8. Close the browser after test execution

Conclusion

I automated the major UI modules of the GitHub Profile Insight application using Java, Selenium WebDriver and TestNG. I used XPath locators, explicit waits and assertions to validate navigation and functionality. This project gave me practical experience in Selenium automation and TestNG-based testing.

Thank You